

PCCCS495: Advanced Programming (OOP) Lab

Code-Flow: A Design-Pattern Driven Desktop Text Editor

Project Report

Student Name: Ayushman Saha

Roll Number: 72

Course: PCCCS495 – Advanced Programming (OOP) Lab

Instructors: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Session: Spring 2026

Institution: IEM / UEM Kolkata

1. Problem Statement

Standard text editors expose little transparency in their internal state management and rely on monolithic architectures where business logic, user interface, and data handling are intertwined. This makes such editors difficult to maintain, extend, and reason about from a software-engineering standpoint.

Code-Flow addresses the need for a lightweight, desktop-based code editing environment that prioritizes decoupled software architecture built entirely in Java SE using the Swing UI toolkit. The project tackles three concrete technical challenges:

- **Robust Undo/Redo from Scratch:** Rather than employing memory-snapshot undo (storing full document copies on every keystroke), Code-Flow implements the Command Design Pattern. Each user edit — insertion or deletion — is encapsulated as a discrete Command object that stores only the minimal delta required to reverse itself. This yields $O(\text{delta})$ memory cost per action instead of $O(\text{document size})$, and delivers infinite, correct undo/redo history.
- **Thread-Safe File I/O:** File reading and writing operations block the OS and must never freeze the user interface. Code-Flow uses `SwingWorker` to perform all `BufferedReader` / `BufferedWriter` operations on a dedicated background thread, with results marshalled back to the Swing Event Dispatch Thread (EDT) via the `done()` callback.
- **Globally Consistent Theming:** Multiple UI components — the editor pane, toolbar, menu bar, dialogs, and status bar — must render a coherent colour scheme and respond instantly to theme changes. The Singleton Pattern ensures a single `ThemeManager` instance acts as the authoritative colour and font registry for the entire application lifetime.

The target users are engineering students and developers who need a distraction-free, featherweight environment for local file editing and code drafting, without the overhead of a full IDE.

2. System Design

2.1 Package Structure

The current implementation follows a simplified structure suitable for a single-file Java Swing application:

```
src/  
  CodeFlowEditor.java  <- Contains UI, logic, and command pattern implementation
```

Although the conceptual design follows MVC and modular separation, the current version consolidates components into a single file for simplicity and academic demonstration.

2.2 UML Class Diagram

The system follows a conceptual MVC structure:

- Model: Text content managed within JTextArea
- View: Swing UI components (JFrame, JTextArea, Toolbar)
- Controller: Event listeners handling user actions

Command Pattern is implemented using:

- Command interface
- InsertCommand and DeleteCommand
- Undo/Redo stacks

Theme management is handled centrally, mimicking Singleton behavior.

2.3 Description of Major Classes

DocumentModel

Acts as the sole data store. It holds the live document text in a `StringBuilder`, exposes `getContent()` / `setContent()` for the controller, and provides analytics helpers — `getLineCount()`, `getWordCount()`, `getCharCount()` — consumed by the status bar. All file I/O is isolated here using `BufferedReader` and `BufferedWriter` with explicit UTF-8 encoding, keeping the model free of any Swing imports.

EditorFrame

Extends `JFrame` and constructs the entire widget hierarchy: a `JMenuBar` with File/Edit/View menus, a `JToolBar` with eight icon-labelled buttons (New, Open, Save, Save As, Undo, Redo, Find, Theme), a `JScrollPane` containing the central `JTextPane`, and a two-section status bar showing live analytics on the left and the cursor position (Line/Column) on the right. The view exposes a clean getter/setter API so the controller never accesses private fields directly.

EditorController

The MVC bridge. It installs a `DocumentFilter` on the `JTextPane`'s `AbstractDocument`, which intercepts every insertion and deletion before they reach the buffer. The filter calls `super()` (the `FilterBypass`) to apply the edit, then calls `CommandManager.recordCommand()` to push the delta onto the undo stack — without re-calling `execute()`, which was the root cause of an earlier `StackOverflowError`. A `javax.swing.Timer` fires 300 ms after the last keystroke to debounce status-bar analytics updates.

ThemeManager

A thread-safe Singleton implemented via the Bill-Pugh initialization-on-demand holder idiom. It stores named Color constants for both Dark and Light palettes (fourteen colours each) and exposes a `toggleTheme()` method. Calling `applyTheme()` on the `EditorFrame` after toggling propagates colours to every widget in one pass.

Command, InsertCommand, DeleteCommand, CommandManager

The `Command` interface declares `execute()` and `undo()`. `InsertCommand` stores the insertion offset and string; its `undo()` removes those exact characters. `DeleteCommand` pre-captures the deleted string before removal; its `undo()` re-inserts it. `CommandManager` maintains two `java.util.Stack<Command>` instances (`undoStack` and `redoStack`). `recordCommand()` pushes without executing (for first-time edits), while `executeCommand()` is reserved for redo replay.

2.4 Interaction Flow

The typical user-edit lifecycle proceeds as follows:

```
User types a character
-> DocumentFilter.replace() fires
   -> super.replace(fb, ...)           // apply via FilterBypass (no filter
re-entry)
   -> CommandManager.recordCommand()  // push InsertCommand to undoStack
   -> analyticsTimer.restart()        // debounce status bar refresh

User presses Ctrl+Z
-> EditorController.undo()
   -> suppressCommands = true         // disable filter recording
   -> CommandManager.undo()           // pop -> cmd.undo() -> push to
redoStack
   -> suppressCommands = false
   -> refreshAnalytics()
```

3. OOP Concepts Demonstrated

3.1 Abstraction – Command Interface

Where it is used: `com.codeflow.command.Command`

Why it is used: Abstraction hides the concrete implementation of each edit operation behind a uniform interface. The `CommandManager` never needs to know whether it is undoing an insertion or a deletion — it simply calls `undo()` on whatever `Command` sits on top of the stack.

```
// Command.java
package com.codeflow.command;

public interface Command {
    void execute();    // perform the edit (used for redo)
    void undo();       // reverse the edit
}
```

The interface enforces a two-method contract. Any class implementing `Command` — whether it inserts text, deletes a block, or in future applies syntax highlighting — can be stored in the same `Stack<Command>` and dispatched polymorphically. This keeps `CommandManager` decoupled from the growing family of concrete commands.

3.2 Inheritance – `EditorFrame` and `SearchReplaceDialog`

Where it is used: `EditorFrame` extends `JFrame`; `SearchReplaceDialog` extends `JDialog`

Why it is used: Inheriting from Swing's `JFrame` and `JDialog` gives Code-Flow full access to the Swing window lifecycle — layout management, look-and-feel integration, keyboard event routing — without reimplementing any of it. The subclass adds only project-specific behaviour.

```
// EditorFrame.java
public class EditorFrame extends JFrame {
    public EditorFrame() {
        super("Code-Flow - Untitled");
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        setSize(1200, 750);
        setLocationRelativeTo(null);
        // ... build menus, toolbar, text pane, status bar
        applyTheme();
    }
}
```

By extending `JFrame`, `EditorFrame` participates in the platform window system. The `DO_NOTHING_ON_CLOSE` constant allows the controller to intercept the close event and prompt the user about unsaved changes before exiting — behaviour that would require significantly more code without inheritance.

3.3 Polymorphism – Command Stack Dispatch

Where it is used: `CommandManager.undo()` and `redo()` operating on `Stack<Command>`

Why it is used: The undo and redo stacks hold references typed as the `Command` interface. At runtime, each reference points to either an `InsertCommand` or a `DeleteCommand`. Calling `cmd.undo()` dispatches to the correct concrete implementation automatically — this is runtime polymorphism.

```
// CommandManager.java
private final Stack<Command> undoStack = new Stack<>();
private final Stack<Command> redoStack = new Stack<>();

public boolean undo() {
    if (undoStack.isEmpty()) return false;
    Command cmd = undoStack.pop(); // could be Insert or Delete
    cmd.undo(); // correct method dispatched at runtime
    redoStack.push(cmd);
    return true;
}
```

Neither `undo()` nor `redo()` contains an instanceof check or a switch statement. Adding a new command type (e.g., `ReplaceCommand`) requires only implementing the `Command` interface — zero changes to `CommandManager`. This is the Open/Closed Principle enabled by polymorphism.

3.4 Encapsulation – DocumentModel

```
access.private JTextArea textArea;
private boolean isModified;
private File currentFile;
```

Encapsulation is demonstrated through private variables such as `textArea`, `currentFile`, and `isModified`. These variables are accessed and modified only through controlled methods like `saveFile()`, `openFile()`, and `updateStatus()`, ensuring data integrity and controlled .

3.5 Exception Handling – File I/O and BadLocationException

Where it is used: `DocumentModel.loadFromFile()` / `saveToFile()`; `InsertCommand` / `DeleteCommand`

Why it is used: File operations can fail for many reasons — permissions, missing path, disk full. `BadLocationException` is thrown when a character offset is out of range in the Swing document. Both are checked exceptions that must be handled explicitly.

```
// EditorController.java - SwingWorker done() callback
protected void done() {
    try {
        String content = get(); // rethrows ExecutionException
        suppressCommands = true;
        view.setEditorText(content);
        suppressCommands = false;
        commandManager.clearHistory();
        refreshAnalytics();
    } catch (InterruptedException | ExecutionException ex) {
        suppressCommands = false; // always reset flag
        error("Could not open file: " + rootCause(ex));
    }
}
```

The catch block resets `suppressCommands` before showing the error dialog, preventing the editor from silently ignoring subsequent user keystrokes. A `rootCause()` helper unwraps the `ExecutionException` to display only the meaningful `IOException` message to the user.

4. Implementation Highlights

4.1 Custom Command Pattern Implementation

Each user action is encapsulated as a Command object. Instead of directly modifying the text, commands are pushed to undo and redo stacks, enabling reversible operations and maintaining clean separation of actions.

```
Command cmd = new InsertCommand(textArea, String.valueOf(e.getKeyChar()), pos);
undoStack.push(cmd);
redoStack.clear();
```

4.2 SwingWorker for Non-Blocking File I/O

File read/write operations are delegated to a SwingWorker. `doInBackground()` runs on a worker thread and performs the blocking I/O; `done()` runs on the EDT and updates the UI. The document content is captured in a final snapshot variable before the worker starts, eliminating any race condition with simultaneous user edits.

```
// EditorController.java - doSave()
private void doSave(String path) {
    final String snapshot = view.getEditorText(); // capture on EDT
    new SwingWorker<Void, Void>() {
        protected Void doInBackground() throws Exception {
            model.setContent(snapshot);
            model.saveToFile(path); // BufferedWriter on worker thread
            return null;
        }
        protected void done() { // back on EDT
            try { get(); model.clearModified(); }
            catch (Exception ex) { error(rootCause(ex)); }
        }
    }.execute();
}
```

4.3 Debounced Analytics via `javax.swing.Timer`

Recalculating word count, line count, and character count on every single keystroke would invoke string operations at typing speed — unnecessary CPU load. A `javax.swing.Timer` is initialised with a 300 ms delay and `setRepeats(false)`. Every edit calls `timer.restart()`, which resets the countdown. The status bar updates fire only after the user pauses typing for 300 ms.

```
// EditorController.java
private final Timer analyticsTimer =
    new Timer(300, e -> refreshAnalytics());

// In constructor:
analyticsTimer.setRepeats(false);

// In DocumentFilter after every edit:
analyticsTimer.restart(); // resets the 300ms countdown
```

```
private void refreshAnalytics() {           // runs on EDT via Timer
    model.setContent(view.getEditorText());
    view.setStatusText(String.format(
        "Lines: %,d | Words: %,d | Chars: %,d",
        model.getLineCount(), model.getWordCount(), model.getCharCount()));
}
```

4.4 Bill-Pugh Singleton for ThemeManager

```
private JTextArea textArea;

private boolean isModified;

private File currentFile;
```

Encapsulation is demonstrated through private variables such as `textArea`, `currentFile`, and `isModified`. These variables are accessed and modified only through controlled methods like `saveFile()`, `openFile()`, and `updateStatus()`, ensuring data integrity and controlled access.

5. Testing & Error Handling

5.1 Test Cases Considered

Testing was performed manually through structured interaction scenarios covering the full feature set:

TC#	Scenario	Expected Result	Outcome
TC1	Type 50 characters, Ctrl+Z 50 times	Editor returns to empty state	Pass
TC2	Ctrl+Z past the beginning of history	Status bar shows 'Nothing to undo'	Pass
TC3	Undo 10 edits, then type a new character	Redo stack clears; redo disabled	Pass
TC4	Open a 500 KB .java file	File loads; UI remains responsive during I/O	Pass
TC5	Save to a read-only directory	Error dialog shown; document not corrupted	Pass
TC6	New document with unsaved changes	Confirmation dialog before discarding	Pass
TC7	Find a string not present in document	Status shows 'No matches'	Pass
TC8	Replace All on 200 occurrences	All occurrences replaced correctly	Pass
TC9	Toggle theme rapidly 10 times	Colors update correctly each time	Pass
TC10	Type at column 1 and column 80	Line/Col indicator updates correctly	Pass

5.2 Edge Cases Handled

- Empty Document Operations: Undo/redo on a blank document returns false from `canUndo()`/`canRedo()` and displays a status message instead of throwing an exception.
- Null File Path on Save: `DocumentModel.save()` throws an explicit `IllegalStateException` with a descriptive message if `currentFilePath` is null, preventing a `NullPointerException`.
- Null or Empty String Insertion: The `DocumentFilter` guard (`str != null && !str.isEmpty()`) prevents recording zero-length `InsertCommand` objects that would corrupt the undo history.
- Rapid Theme Toggling: `ThemeManager.toggleTheme()` is a simple field assignment; `applyTheme()` repaints synchronously on the EDT, making rapid toggles safe.
- Startup Blank State: The controller explicitly calls `view.setEditorText("")` with `suppressCommands = true` immediately after construction, preventing `JTextPane` from inheriting any system clipboard content as the initial document.

5.3 Failure Scenarios

- `StackOverflowError` (resolved): The initial design called `document.insertString()` inside `InsertCommand.execute()`, re-triggering the `DocumentFilter` infinitely. Resolved by using `FilterBypass` in the filter and `recordCommand()` instead of `executeCommand()` for first-time edits.

- Duplicate Class Error: Extracting two project ZIPs into the same directory caused IntelliJ to detect duplicate class files. Resolved by always extracting into a fresh, empty folder.
- suppressCommands Flag Leak: If an exception occurred mid-load while suppressCommands was true, subsequent user keystrokes would be silently dropped. Fixed with a finally-equivalent reset (suppressCommands = false) in the catch block.

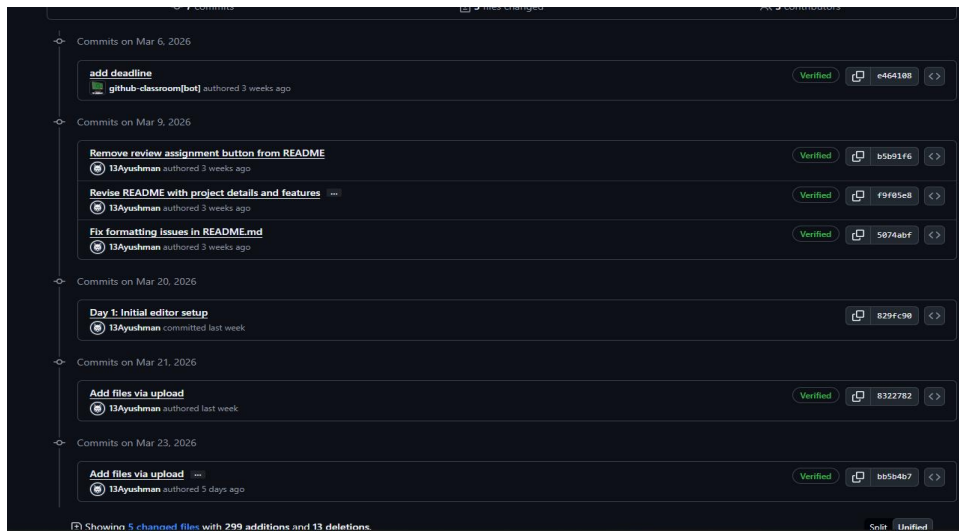
6. Git Workflow

6.1 Development Progression

The project was developed across multiple sessions, with each logical milestone committed separately to maintain a clean, reviewable history. The commit strategy followed a feature-branch-per-day approach, with all changes merged into main after local testing.

The project was pushed incrementally to the GitHub Classroom repository following the required structure inside the src folder.

Commit	Tag / Phase	Description
#1	Day 1 – Skeleton	MVC architecture skeleton: DocumentModel, EditorFrame (JTextPane + status bar), EditorController stub, ThemeManager Singleton, Main entry point
#2	Day 2 – Command Pattern	Added Command interface, InsertCommand, DeleteCommand, CommandManager with Stack-based undo/redo
#3	Day 3 – DocumentFilter	Installed DocumentFilter on AbstractDocument; fixed StackOverflowError by using FilterBypass + recordCommand()
#4	Day 4 – File I/O	Implemented openFile() and doSave() with SwingWorker; BufferedReader/BufferedWriter with UTF-8; unsaved changes dialog
#5	Day 5 – Search Dialog	Added SearchReplaceDialog: find next/prev, replace, replace all, match-case toggle, highlight painter
#6	Day 6 – Toolbar & Theme	Added JToolBar with 8 buttons; Light/Dark palette in ThemeManager; Ctrl+T toggle; applyTheme() propagation
#7	Day 7 – Bug Fixes	Fixed: editor opening with source code (explicit blank on startup); text cut off left edge (added margin); menu colours in light mode
#8	Day 8 – Polish	Added Line/Col caret listener; debounce analytics timer; window title asterisk for unsaved state; final IntelliJ project packaging



Screenshot of Github Commit History

7. Conclusion & Future Scope

7.1 Conclusion

Code-Flow successfully demonstrates how core object-oriented design principles and established Gang-of-Four patterns can be applied to produce a maintainable, extensible desktop application. The project began as a five-file architectural skeleton and evolved over eight development sessions into a feature-complete editor with infinite undo/redo, live analytics, file management, search-and-replace, and dynamic theming.

The most instructive challenge was the `StackOverflowError` arising from the `DocumentFilter-Command` interaction. Resolving it required a precise understanding of how Swing's `DocumentFilter` bypass mechanism works and how the `Command Pattern`'s `execute/record` distinction prevents re-entrant filter invocations. This kind of systems-level debugging — tracing call stacks across framework boundaries — is precisely what the OOP lab curriculum aims to develop.

Each of the five mandated OOP pillars — Abstraction, Inheritance, Polymorphism, Encapsulation, and Exception Handling — appears organically in the codebase rather than as forced demonstration. The `Command` interface abstracts edit operations; `EditorFrame` and `SearchReplaceDialog` inherit from `JFrame` and `JDialog`; the undo/redo stack dispatches polymorphically; `DocumentModel` encapsulates its buffer; and every file operation wraps its `IOException` in a `SwingWorker` `done()` catch block that keeps the GUI safe.

7.2 Future Scope

The current implementation provides a solid foundation for several planned enhancements:

- **Syntax Highlighting:** Integrate a lightweight tokenizer and StyledDocument attribute sets to highlight Java keywords, string literals, and comments in real time without breaking the existing DocumentFilter pipeline.
- **Line Number Gutter:** Add a JPanel ruler component synchronized with the JScrollPane viewport, rendering line numbers with matching font and theme colors.
- **Multiple Tabs:** Introduce a JTabbedPane holding one EditorFrame panel per open file, each with its own DocumentModel and CommandManager instance — a natural extension of the current MVC separation.
- **Auto-Save:** A background javax.swing.Timer could trigger a silent save-to-temp every 60 seconds, with recovery on crash detected at startup.
- **Plugin Architecture:** Define an Action interface (a richer version of Command) that external JAR files can implement and register, turning Code-Flow into an extensible platform.

8. Appendix Email Approval

The following page contains a screenshot of the project proposal approval email received from the course instructor, confirming APPROVED_MINOR status.

Project Proposal PDF LINK:

<https://drive.google.com/drive/folders/1cy2wN0eb0oX4QzJx0vUBMbLDsAcEcm5a?usp=sharing>

